PHISHSHIELD

Machine Learning Based Phishing URL Detection

CLOUD PRODUCT AND PLATFORM ENGINEERING

PROJECT REPORT

Submitted by

Vaishnavi [RA2311056030134]
Manisha Kumari [RA2311056030187]
Adarsh Vijay [RA2311056030190]
Prince Panwar [RA2311056030142]

Under the Guidance of

Dr. M. Sivakumar
Assistant Professor
Department of Networking and Communications

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
with specialization in Data Science



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

OCTOBER 2026

ABSTRACT

PhishShield is a machine learning application that estimates phishing risk from the structure of a URL. It combines a reproducible Python training pipeline, MLflow experiment tracking, a local Streamlit interface and a public browser application on Hugging Face Spaces. The project demonstrates how model development, software testing, release verification and deployment can be connected in an academic cloud product workflow.

The application uses 30 lexical features and does not open submitted websites. A cleaned Hannousse–Yahiouche dataset of 11,411 URLs is divided into separate domain groups for training, model selection, calibration and final testing. Logistic Regression, Random Forest and Histogram Gradient Boosting are compared on validation data. The selected 200-tree Random Forest is calibrated with a sigmoid function and evaluated on 1,944 held-out URLs. It achieves 82.00% accuracy, 86.49% phishing precision, 75.80% phishing recall and an F1 score of 80.79%.

The deployed Static Space runs the exported model locally in the browser using Pyodide and a background worker. A comparison of 11,980 valid and invalid inputs passes with no mismatched outcomes and a maximum probability difference of 3.33×10^{-16} relative to the Python reference. Jira provides task visibility, GitHub stores the source, and Jenkins executes the local verification pipeline. Model warnings and measurable limitations are presented alongside the prediction to support informed review.

PhishShield is an academic decision aid. Its historical evaluation does not establish the safety of a current website, and a low model score cannot rule out phishing. The final system demonstrates a working deployment with traceable evidence while identifying current-data evaluation and automated release governance as future work.

Keywords: phishing detection, URL analysis, Random Forest, MLflow, Jenkins, Streamlit, Hugging Face Spaces, Pyodide.

TABLE OF CONTENTS

Section	Title	Page
	ABSTRACT	i
	LIST OF FIGURES	v
	LIST OF TABLES	vi
	ABBREVIATIONS	vii
1	**INTRODUCTION**	1
1.1	Project Background	1
1.2	Problem Statement	1
1.3	Need for the Project	1
1.4	Objectives	2
1.5	Scope of the Project	2
1.6	Organization of the Report	2
2	**LITERATURE SURVEY AND EXISTING SYSTEM**	3
2.1	Traditional Development Process	3
2.2	Challenges in Existing Manual Systems	3
2.3	DevOps and Agile Methodologies	3
2.4	Review of Tools	4
3	**REQUIREMENT ANALYSIS**	5
3.1	Functional Requirements	5
3.2	Non Functional Requirements	5
3.3	Hardware Requirements	6
3.4	Software Requirements	6
4	**SYSTEM DESIGN**	7
4.1	Overall Architecture	7
4.2	Workflow Diagram	8

TABLE OF CONTENTS

Section	Title	Page
4.3	Database Design	8
4.4	CI and CD Pipeline Design	8
5	**PROJECT SETUP AND CONFIGURATION**	9
5.1	Jira Setup	9
5.2	GitHub Setup	10
5.3	Jenkins Setup	11
5.4	MLflow Setup	11
6	**IMPLEMENTATION**	13
6.1	Module Development	13
6.2	Source Code Management	14
6.3	Jenkins Pipeline Implementation	14
6.4	ML Model Training and Logging	14
6.5	Deployment Process	15
6.6	Screenshots of the Application	16
7	**TESTING**	18
7.1	Testing Strategy	18
7.2	Unit Testing	18
7.3	Integration Testing	18
7.4	System Testing	18
7.5	CI Pipeline Testing	19
7.6	Performance Testing	19
7.7	Defect Tracking Using Jira	19
7.8	Test Cases	19
8	**RESULTS AND DISCUSSION**	20

TABLE OF CONTENTS

Section	Title	Page
8.1	Automation Results	20
8.2	Build Success Rate	20
8.3	Reduced Deployment Time	20
8.4	Collaboration Improvements	20
8.5	MLflow Experiment Tracking Results	21
9	**CHALLENGES FACED**	25
9.1	Dataset Bias and Generalization	25
9.2	Deployment Constraints	25
9.3	Permissions and Integration	25
9.4	Verification and Interpretation	25
10	**CONCLUSION**	26
11	**FUTURE ENHANCEMENTS**	27
11.1	Evaluation on Newer Data	27
11.2	Release Automation and Quality	27
11.3	Runtime and Monitoring	27
11.4	Detection and Usability	27
	REFERENCES	28
	APPENDICES	29
	Appendix A Jira Workflow and Team	29
	Appendix B Git and Reproduction Commands	29
	Appendix C Jenkinsfile and Pipeline	30
	Appendix D Verification Records	31
	Appendix E Screenshot Index	31
	Appendix F Code and Release Identity	32

LIST OF FIGURES

Figure	Title	Page
4.1	Training and deployment architecture	7
4.2	Verification workflow and release gate	8
5.1	Jira board with assigned PhishShield work	9
5.2	Jira summary showing nine tracked work items	10
5.3	Successful Jenkins build and archived verification reports	12
6.1	Public Hugging Face Static Space URL input	16
6.2	Local Streamlit result for a misleading address	17
8.1	MLflow candidate and historical run comparisons	21
8.2	Held-out confusion matrix for the deployed release	22
8.3	MLflow verification metrics and historical accuracy runs	23
8.4	Model Insights on the running Hugging Face Space	24

LIST OF TABLES

Table	Title	Page
2.1	Tools and their project roles	4
3.1	Functional requirements	5
3.2	Software environment	6
6.1	Implementation modules	13
6.2	Actual domain-disjoint data partitions	14
7.1	Representative functional and pipeline tests	19
8.1	Candidate comparison on 2,011 validation URLs	21
8.2	Final calibrated model on 1,944 held-out URLs	22
A.1	Project team	29
C.1	Jenkins pipeline stages	30
D.1	Verification evidence	31
F.1	Reproducibility identifiers	32

ABBREVIATIONS

Abbreviation	Meaning
AP	Average Precision
API	Application Programming Interface
CI	Continuous Integration
CD	Continuous Delivery or Deployment
CSE	Computer Science and Engineering
FN	False Negative
FP	False Positive
FPR	False Positive Rate
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP over Transport Layer Security
IDNA	Internationalized Domain Names in Applications
JSON	JavaScript Object Notation
ML	Machine Learning
PSL	Public Suffix List
ROC AUC	Area under the Receiver Operating Characteristic curve
SCM	Source Code Management
SHA-256	Secure Hash Algorithm with a 256-bit digest
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
WASM	WebAssembly

CHAPTER 1

INTRODUCTION

1.1 Project Background

Phishing links imitate familiar services or use misleading address structures to persuade users to disclose credentials or other sensitive information. A URL can contain useful warning signs before a page is opened: an IP address in place of a domain, user information before an @ symbol, unusual hostname spelling, or a long path containing account-related terms. These patterns are signals for review, not definitive proof of malicious intent.

PhishShield turns this problem into a complete software and machine learning project. The same feature definitions support training and inference. The development workflow records data provenance, compares candidate models, exports a verifiable release and exposes its predictions through an accessible interface. Source control, task tracking and automated checks make the result easier to reproduce and review [1].

1.2 Problem Statement

A useful phishing demonstrator must do more than display a label. It must validate input consistently, report the limitations of URL-only evidence, avoid mixing training and test data, and show that deployment has preserved the original model. Manual copying of weights or separate feature implementations can create differences between development results and the public application.

1.3 Need for the Project

The project addresses the need for a transparent and reproducible URL risk analyzer. It also provides a practical setting for cloud product engineering: four team members can track work in Jira, maintain code in GitHub, examine experiments in MLflow and use Jenkins to check a release. A browser deployment makes the demonstration accessible without operating a prediction server.

1.4 Objectives

The main objective is to deliver a functioning phishing risk application whose deployed predictions agree with a tested Python model. The supporting objectives are to prepare a traceable labeled dataset, extract consistent URL features, compare three candidate classifiers, calibrate the selected model, expose evaluation evidence, and preserve reproducible artifacts.

The engineering objectives are to organize project tasks, maintain versioned source, execute automated tests, verify model integrity and package a deployable release. The public interface must clearly distinguish the statistical model estimate from independent domain warnings and must avoid visiting the submitted address.

1.5 Scope of the Project

The implemented scope covers HTTP and HTTPS URL text, offline parsing, 30 lexical features, binary classification, calibrated probability, risk bands and a limited catalogue of domain-lookalike warnings. The local interface is written in Streamlit. The deployed interface is a Static Space with browser inference; it does not run a Streamlit server.

The project excludes webpage content inspection, DNS queries, WHOIS records, certificate inspection, redirect following and live reputation services. It does not store submitted URL history. The training corpus is historical, and the reported metrics apply to its held-out domain groups. Docker configuration is available as an alternative deployment package; the verified public release uses the Static Space architecture.

1.6 Organization of the Report

Chapter 2 discusses related approaches and engineering tools. Chapters 3 and 4 specify requirements and design. Chapters 5 and 6 explain setup, implementation and deployment. Chapters 7 and 8 present testing and measured results. Chapters 9 to 11 describe challenges, conclusions and future enhancements. The appendices provide commands, pipeline details, test evidence and release identifiers.

CHAPTER 2

LITERATURE SURVEY AND EXISTING SYSTEM

2.1 Traditional Development Process

A manually assembled ML demonstration often separates the notebook used for training from the application used for prediction. If preprocessing is copied into a second code path, a change to normalization or feature ordering can silently alter results. Dependencies and model files may also differ across computers. This project therefore treats the feature schema, fitted estimator and release metadata as one versioned unit.

2.2 Challenges in Existing Manual Systems

URL analysis involves a trade-off between available evidence and runtime cost. Blacklists can identify previously recorded addresses, whereas lexical classifiers infer patterns from labeled examples. Content-based systems can examine page behavior, but they require additional collection and execution infrastructure. PhishShield restricts itself to URL text so that analysis can run locally without contacting the submitted host.

Hannousse and Yahiouche publish a balanced corpus of 11,430 URLs with URL, page-content and external-service features [2]. PhishShield uses only the raw URL and status label and recomputes its own lexical features. The source dataset was collected in May 2020. Its age and collection method limit conclusions about current attacks.

2.3 DevOps and Agile Methodologies

For this project, Agile practices provide a visible task workflow and ownership, while CI provides repeatable verification. The Jira board uses To Do, In Progress, In Review and Done columns. GitHub provides a shared source location. Jenkins executes checks against a source snapshot and publishes reports. MLflow records model experiments independently of the user interface, supporting traceability between a training run and an exported release.

2.4 Review of Tools

Table 2.1 Tools and their project roles

Tool	Role in PhishShield
Jira	Task assignment, board status and review visibility.
Git and GitHub	Versioned Python source, tests and workflow configuration.
Jenkins	Local pipeline for tests, model verification, parity checks and packaging.
MLflow	Candidate runs, metrics, dataset lineage, artifacts and model registry.
scikit-learn	Classifier training, model comparison and probability calibration.
Streamlit	Local interactive URL analyzer and model insights.
Hugging Face Spaces	Public hosting of the static application and release assets.
Pyodide	Python runtime inside a browser worker for inference.

MLflow separates run metadata from large artifacts and supports tracking parameters and metrics for model comparison [4]. A probability calibration stage is needed because a classifier's raw score need not be a reliable probability estimate. The selected model is frozen before its sigmoid calibrator is fitted on a separate partition, consistent with the independence requirement described by scikit-learn [3].

The repository contains GitHub Actions test configuration as well as the Jenkins pipeline added for the academic demonstration. Bitbucket is not used. Jenkins and GitHub Actions provide related CI capabilities, but the measured Jenkins runs in this report are distinct from any hosted GitHub Actions execution [6], [7].

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Functional Requirements

Table 3.1 Functional requirements

ID	Requirement	Acceptance condition
FR1	Validate URL input	Reject missing schemes, malformed hosts, unsupported ports and invalid text.
FR2	Extract consistent features	Use the same ordered 30-feature schema in training and inference.
FR3	Estimate phishing risk	Return calibrated probability, model class and a risk band.
FR4	Show contextual warnings	Display structural indicators and separate domain-lookalike warnings.
FR5	Present model evidence	Expose held-out metrics and evaluation plots.
FR6	Track model development	Record dataset provenance, candidate comparisons and release identity.
FR7	Verify and package releases	Tests, model integrity and parity checks must pass before packaging.
FR8	Provide a public demo	Load the Static Space and analyze URLs in the browser.

3.2 Non Functional Requirements

Privacy requires that analysis does not visit a submitted website or send the submitted URL to a prediction service. Reliability requires deterministic feature order, pinned dependencies, verified artifacts and understandable input errors. Maintainability requires separate modules for parsing, training, inference and presentation. The interface must remain responsive while the browser runtime is loading or evaluating a URL.

3.3 Hardware Requirements

Development and the recorded tests use a local macOS workstation. Training is CPU-based; the project does not require a GPU. A supported Java installation runs Jenkins, and a modern browser runs the Static Space. Internet access is needed to download dependencies and the browser runtime on first use. Once loaded, URL evaluation occurs on the client device.

Memory, cold-start latency and maximum concurrent-user capacity were not benchmarked. A practical development machine needs enough disk space for the Python environment, training data, model artifacts and Jenkins workspace. These are operational requirements rather than measured minimum hardware specifications.

3.4 Software Requirements

Table 3.2 Software environment

Component	Recorded configuration
Python runtime	3.13.2 for model training and inference; source supports 3.12 and 3.13.
Machine learning	scikit-learn 1.9.1, NumPy 2.5.4, pandas 2.3.3, joblib 1.6.0.
Experiment tracking	MLflow 3.17.1 with local tracking database and artifact storage.
URL processing	tldextract 5.4.0 with a fixed suffix snapshot; IDNA handling.
Local app	Streamlit with Plotly and matplotlib visualizations.
Browser inference	Pyodide 0.29.3, CPython 3.13.2, HTML, CSS and JavaScript.
CI runtime	Jenkins 2.580.1 with OpenJDK 25.0.1.
Quality assurance	pytest, dependency consistency checks and exported prediction fixtures.

CHAPTER 4

SYSTEM DESIGN

4.1 Overall Architecture

The system has two paths. The development path prepares data, trains and selects a model, calibrates it, records the experiment and exports a release. The inference path accepts a URL, validates it, extracts features, evaluates the fitted forest and presents the calibrated probability with contextual indicators. Both paths use feature schema 1.1.0.

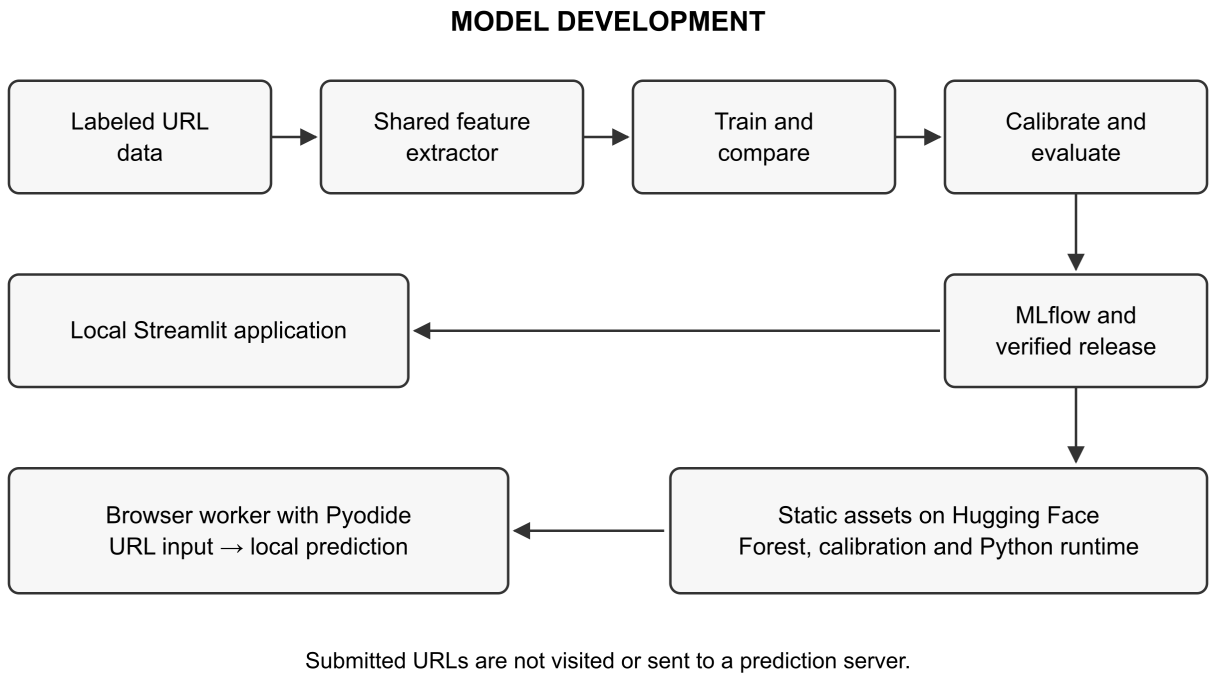


Figure 4.1 Training and deployment architecture

The local Streamlit app loads the serialized Python release. The public Static Space loads an exported tree representation and runs a compatible evaluator in a Pyodide worker. The browser bundle includes domain parsing resources and calibration parameters. MLflow is needed for development and review, but the public application does not depend on a running MLflow server.

4.2 Workflow Diagram

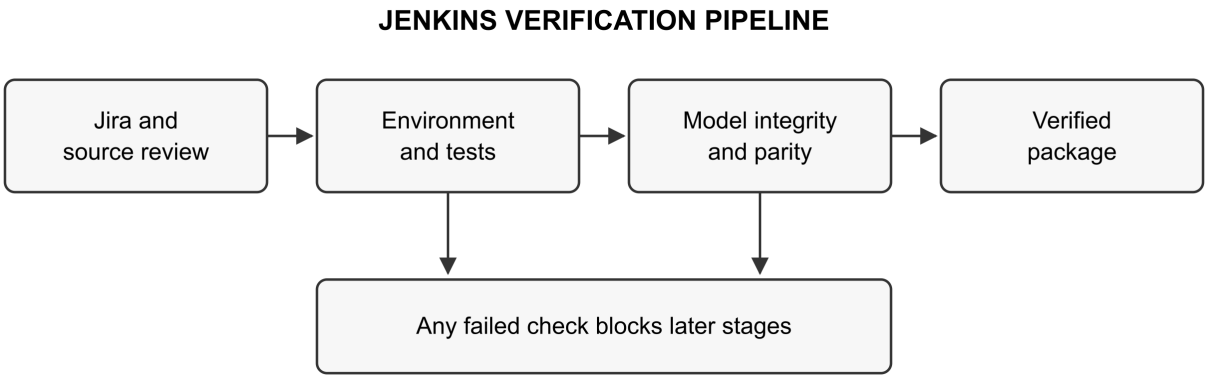


Figure 4.2 Verification workflow and release gate

Jira work items identify changes to review. The source snapshot then passes through Jenkins environment checks, tests, release verification and export parity checks. Packaging is reached only if earlier stages pass. The deployed release was published through the authorized Hugging Face account and tested on the live Space. A push-triggered production deployment has not been configured.

4.3 Database Design

The application does not require a database of submitted URLs or user accounts. During development, MLflow stores run and registry metadata locally in mlflow.db and keeps artifacts in mlartifacts. CSV and JSON files preserve dataset summaries, split assignments, comparisons and release metadata. The browser uses downloaded static resources and transient input state.

4.4 CI and CD Pipeline Design

The Jenkins job accepts explicit paths for the source, locked Python environment and static release. It works on a copied source snapshot and publishes a JUnit report, verification JSON, parity JSON and a release ZIP. A controlled failure parameter demonstrates that a failed stage blocks downstream packaging. The pipeline is a local CI and release-verification workflow; external webhooks and unattended cloud deployment remain extensions.

CHAPTER 5

PROJECT SETUP AND CONFIGURATION

5.1 Jira Setup

The team uses the Jira space named Wipro Project. Its board organizes PhishShield work under To Do, In Progress, In Review and Done. Visible tasks include repository verification, pipeline review, prediction testing, and demonstration documentation. The board associates work with the PhishShield epic and uses phishshield and wipro labels.

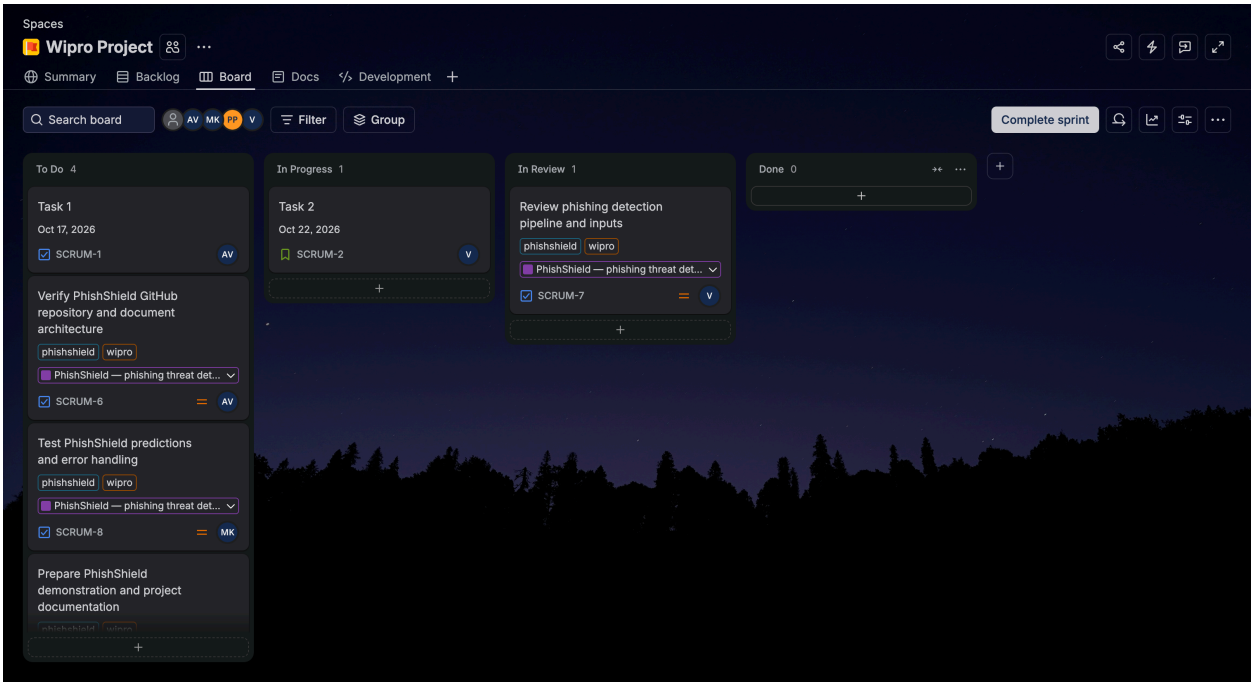


Figure 5.1 Jira board with assigned PhishShield work

The captured board shows work at different stages and no completed cards. It records the actual state at capture time and should not be read as a completed sprint. Repository verification is assigned to Adarsh Vijay, pipeline review to Vaishnavi and prediction testing to Manisha Kumari. The screenshots do not establish an exclusive contribution split for every team member.

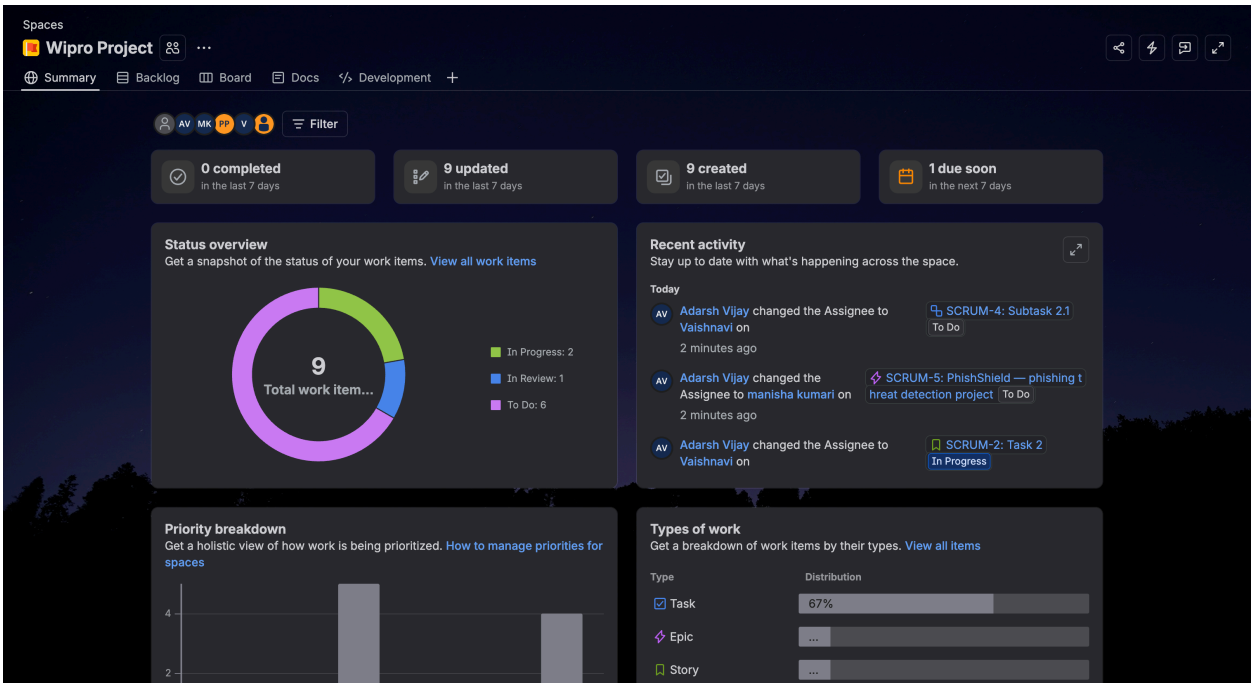


Figure 5.2 Jira summary showing nine tracked work items

The summary view contains nine work items: six To Do, two In Progress and one In Review, with zero completed. It also shows nine items created and updated in the preceding seven days. The summary covers a different view of work than the six visible top-level board cards, so their counts should not be substituted for each other.

5.2 GitHub Setup

The source repository is Adarshvijaycx/PhishShield on GitHub [1]. It contains the application, Python package, tests, dependency locks and supporting documentation. The deployment work started from revision 7662eb555af538a40a7aa0202f689f0f1752699b. Generated data, local tracking state and training outputs are handled separately from ordinary source files.

The local Jenkinsfile and CI helper scripts provide the pipeline implementation documented here. A release review should record both the repository revision and any local changes used for the build. The live Space revision is a separate identifier and is recorded in Appendix F.

5.3 Jenkins Setup

Jenkins 2.580.1 is installed from the official WAR distribution and its SHA-256 digest is checked. OpenJDK 25.0.1 runs the controller. The controller binds to 127.0.0.1 on port 8088, stores its state within the project workspace and uses an administrator account. Read access to the local demonstration is available without exposing administrative permissions.

The PhishShield-CI job uses a Declarative Pipeline with one executor. Pipeline, JUnit and stage-view plugins support execution, test reporting and visual inspection. The job takes SOURCE_DIR, PYTHON_BIN and STATIC_DIR parameters. INJECT_FAILURE is reserved for a controlled negative test. Local paths are supplied when starting a build; they are not cloud credentials.

5.4 MLflow Setup

The PhishShield experiment records candidate and calibrated-winner runs. The local tracking service uses port 5001. Metrics, charts, dataset lineage and registry information are saved for review. The source project's screenshot shows registry version 3; a fresh training database used for deployment starts at version 1. Registry numbers are local identities, so the deployment is identified by its run and artifact checksum as well as its metrics.

Jenkins build #2 completed the full workflow in 30.137 seconds. All 217 tests passed in 18.76 seconds, twenty saved release examples verified, and the 11,980-case parity comparison had no failures. Build #1 was the deliberate negative control. Its failed environment stage prevented packaging, as intended.

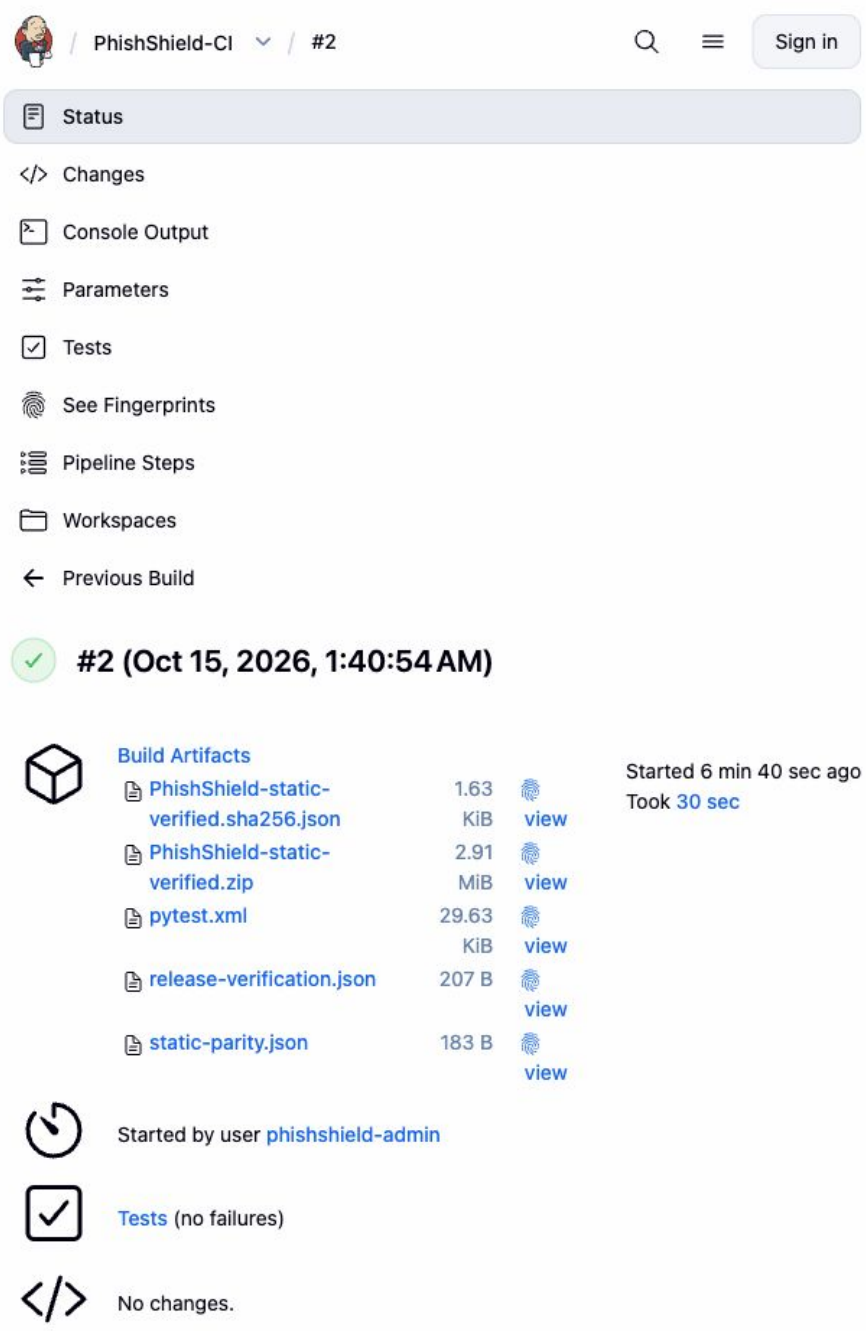


Figure 5.3 Successful Jenkins build and archived verification reports

CHAPTER 6

IMPLEMENTATION

6.1 Module Development

Table 6.1 Implementation modules

Module	Responsibility
features.py	Offline validation, normalization, registered-domain extraction and 30 numeric features.
domain_signals.py	Independent lookalike-domain and misleading-hostname warnings.
data.py	Dataset acquisition, provenance, cleaning and grouped partitions.
training.py	Candidate training, selection, calibration, evaluation and export.
inference.py	Load and verify a release, evaluate URLs and format results.
cli.py	Download, train, verify, analyze, serve and review commands.
app.py	Local Streamlit input, predictions and model evidence.
worker.js and browser bundle	Background Python inference for the public Static Space.

Validation checks that an input is text, contains an HTTP or HTTPS scheme and has a valid hostname and port. URLs longer than 8,192 characters, control characters, unencoded whitespace and backslashes are rejected. The parser normalizes the scheme and hostname, handles international domain names and canonicalizes an empty root path and a single root slash consistently.

The fixed Public Suffix List includes private suffixes and avoids network updates during inference. Lexical features describe lengths, counts, ratios, paths, query parameters, keywords, HTTPS, IP hosts, user information, ports, punycode and hostname entropy.

6.2 Source Code Management

The repository separates reusable model logic from presentation. Dependency lock files define the environment, and tests exercise the same code paths used for prediction. The static exporter retains the original parser and domain-warning behavior while replacing dependencies needed only for training. The export keeps the feature order and the fitted calibration parameters.

6.3 Jenkins Pipeline Implementation

The pipeline copies the source into its own workspace, checks dependencies and compiles Python files, runs pytest with JUnit output, verifies the trained release, compares static inference against reference fixtures, and packages the verified static files. Reports and the ZIP are archived after execution. An intentional failure stops later stages; a successful verification is required to reach packaging. Appendix C lists the implemented stages and key commands.

6.4 ML Model Training and Logging

The pipeline recombines the pinned dataset mirror partitions and retains only URL text and labels. Eight invalid rows and eleven duplicates are removed, leaving 11,411 rows across 7,259 domain groups. Domain-disjoint partitions use seed 43. Group sizes cause actual row proportions to differ from the requested 60/15/10/15 split.

Table 6.2 Actual domain-disjoint data partitions

Partition	URLs	Legitimate	Phishing	Domain groups
Training	6,158	3,304	2,854	4,355
Validation	2,011	802	1,209	1,089
Calibration	1,298	631	667	726
Test	1,944	973	971	1,089
Total	11,411	5,710	5,701	7,259

Candidate models use balanced class weights. Logistic Regression provides a linear reference. Random Forest captures nonlinear feature interactions through an ensemble of trees. Histogram Gradient Boosting offers a second nonlinear candidate. Validation average precision selects the winner, with phishing recall as the tie-breaker. Test metrics are reserved for the final calibrated model.

The chosen forest contains 200 trees, has a maximum depth of 20 and uses a minimum leaf size of 2. After fitting on training data, the estimator is frozen and sigmoid calibration is fitted using the separate calibration set. The final threshold is 0.5. A score below 30 is Low, from 30 to below 70 is Medium, and at least 70 is High. These display bands are distinct from the binary classification threshold.

6.5 Deployment Process

A Docker and Streamlit package was prepared first. The final deployment uses a free Static Space at Adarsh-Vijay/PhishShield. Hugging Face serves the HTML, JavaScript, styling, model bundle and evidence images. A Web Worker loads Pyodide and the exported predictor, so model evaluation occurs in the browser [5], [8].

The export stores the forest structure, leaf probabilities and original sigmoid calibration. Feature values follow the reference float32 conversion, while tree thresholds and probabilities retain their numeric precision. The browser uses the same Python parsing behavior to avoid differences caused by a separate JavaScript URL parser. Each result returns the model score, class, effective hostname, indicators and independent caution assessment.

The published files were verified against local asset digests, and prediction parity was tested on the live Space. The app does not upload the entered URL or follow it. The first visit still requires downloading the runtime and model assets. Browser inference is independent of the local Streamlit and MLflow processes.

6.6 Screenshots of the Application

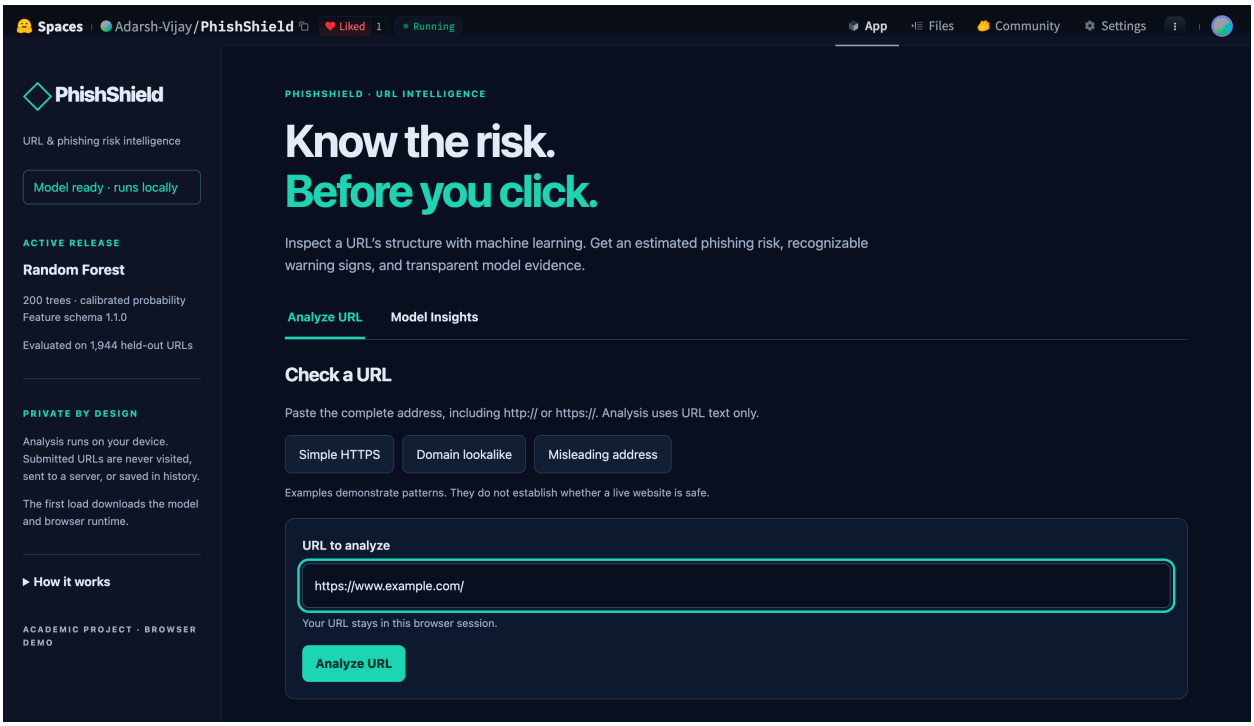


Figure 6.1 Public Hugging Face Static Space URL input

The deployed interface provides an Analyze URL tab, a Model Insights tab and example inputs. Its sidebar identifies the active model as a calibrated 200-tree Random Forest and describes the local browser execution. The Hugging Face header shows the Space in the Running state at capture time.

Example buttons illustrate URL patterns; they do not certify that a live website is safe. The input message explains that the URL remains in the browser session. While the runtime loads, the interface reports model status before enabling analysis. This makes the initialization phase visible to the user.

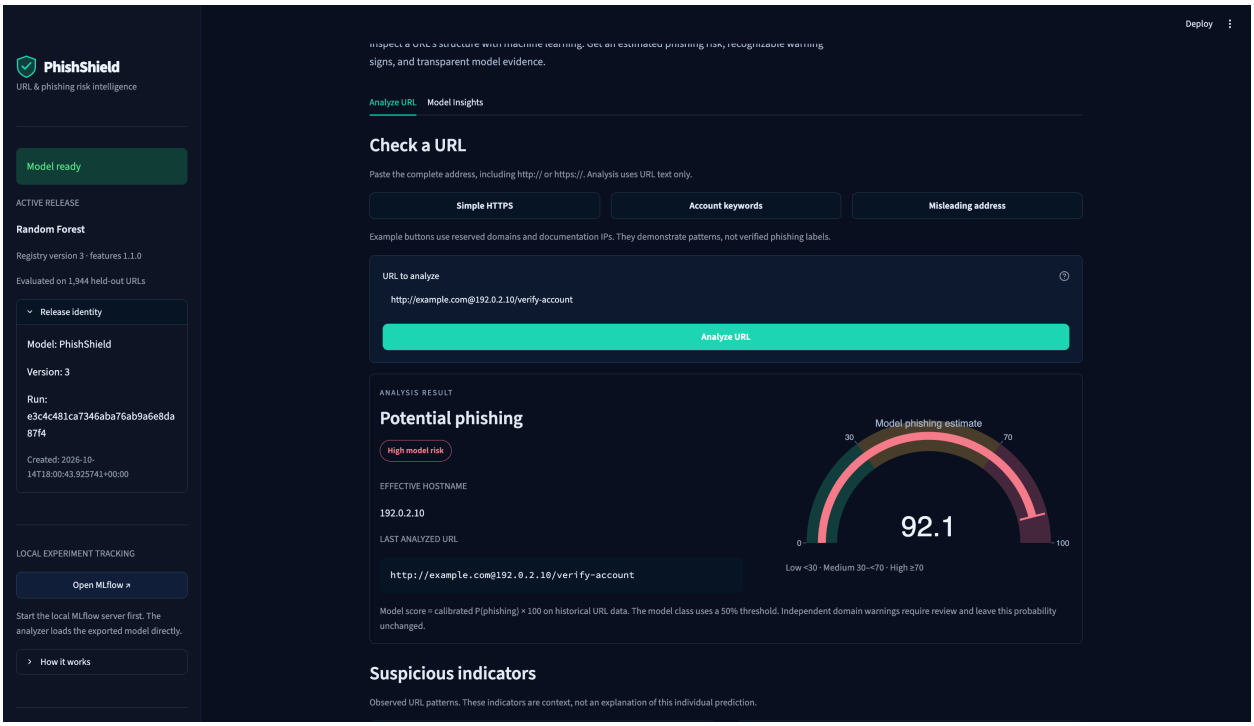


Figure 6.2 Local Streamlit result for a misleading address

The local Streamlit screenshot analyzes a URL whose visible user-information portion contains example.com but whose effective host is 192.0.2.10. The result is Potential phishing with a score of 92.1 and High model risk. The address uses a documentation IP for demonstration, and the application does not visit it.

A model probability and a domain warning serve different purposes. The probability comes from the fitted classifier and calibrator. A warning highlights a recognizable spelling or address pattern. PhishShield keeps that warning separate instead of modifying the numeric estimate, which makes the displayed result easier to audit.

CHAPTER 7

TESTING

7.1 Testing Strategy

Testing covers data preparation, URL parsing, feature stability, inference, interface behavior, model tracking and release integrity. The export adds a second layer: the browser-compatible evaluator must agree with Python on both valid inputs and rejected inputs. Test evidence is separated from statistical model accuracy.

7.2 Unit Testing

Unit tests exercise malformed addresses, Unicode and IDNA handling, IP hosts, trailing dots, ports, query strings, features and domain signals. They check deterministic normalization and ensure warning rules do not silently alter the model probability. The earlier local suite contains 217 passing tests. Jenkins build #2 repeats all 217 tests successfully in 18.76 seconds and publishes its JUnit report.

7.3 Integration Testing

Integration tests cover grouped data splits, candidate training, calibration, release loading and MLflow tracking or registry behavior with small fixtures. The full release verifier checks model integrity and twenty saved reference examples. Streamlit tests verify visible results and error handling, including the separation between low model risk and high caution caused by a lookalike domain.

7.4 System Testing

System checks use the actual trained release in the local app and the published Static Space. The live comparison includes 11,980 inputs: 11,893 accepted URLs and 87 invalid inputs. It compares all 30 features, normalized URLs, host/domain values, probabilities, categories, bands, indicator codes, domain-warning codes and validation errors. All cases pass with probability tolerance 10^{-12}.

7.5 CI Pipeline Testing

The Jenkins negative-control run intentionally raises an error after environment verification. Its downstream testing, release verification, parity and packaging stages must be skipped. Build #1 fails intentionally and skips those stages. Build #2 completes the normal workflow successfully in 30.137 seconds and archives its evidence. This demonstrates that the configured quality gate is active; it is not a measurement of long-term build reliability.

7.6 Performance Testing

The earlier full local test suite completed in 51.43 seconds, and the static export completed the parity dataset without mismatch. These are validation observations, not browser latency or load benchmarks. Cold-start download time, per-URL latency across devices and concurrent hosting performance require a separate controlled experiment.

7.7 Defect Tracking Using Jira

Jira includes work items for pipeline review and prediction/error testing. A reproducible defect record should identify the input, expected behavior, observed output, affected release and verification result. The captured board shows these work items pending or under review; it does not establish that every defect has been closed.

7.8 Test Cases

Table 7.1 Representative functional and pipeline tests

ID	Module	Input or check	Expected output	Result
TC01	Parser	Missing HTTP(S) scheme	Clear validation error	Pass
TC02	Parser	Invalid host, port or whitespace	Reject consistently	Pass
TC03	Features	Equivalent root slash forms	Identical feature values	Pass
TC04	Inference	Misleading @ and IP address	Effective IP host; high model risk	Pass
TC05	Warnings	mail.g0ogle.com	Lookalike warning; original score retained	Pass
TC06	Release	Saved model and fixtures	Integrity and 20 examples verified	Pass
TC07	Export	11,980 reference inputs	No outcome differences	Pass
TC08	Jenkins	Controlled failure switch	Later stages blocked	Pass

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Automation Results

The project produces a repeatable path from verified source and a trained release to a static package. Jenkins archives test and verification artifacts, and the deployed model operates without a Python prediction server. Native, local-browser and live-Space comparisons establish agreement on the tested inputs. Six hosted core assets also matched the local SHA-256 values.

8.2 Build Success Rate

The Jenkins demonstration includes a deliberately failed gate test followed by a successful verification run. Its build history is reported with the purpose of each run in Appendix D. A percentage from this small, intentionally mixed set would not estimate normal build reliability. A useful success-rate measure requires a longer history of ordinary builds with deliberate negative tests identified separately.

8.3 Reduced Deployment Time

The Static Space removes the need to maintain a running Python prediction service for this demonstration. Jenkins packages the files after verification, reducing manual repetition in the release process. No matched before-and-after timing experiment was performed, so the report does not assign a percentage improvement to deployment time.

8.4 Collaboration Improvements

Jira exposes ownership and review state, GitHub provides source history, and MLflow provides experiment evidence. Together, these tools make it possible to discuss a specific task, code revision and model release. The screenshots demonstrate shared work visibility, but they do not quantify a productivity gain or show a completed sprint.

8.5 MLflow Experiment Tracking Results

Table 8.1 Candidate comparison on 2,011 validation URLs

Candidate	Validation AP	Validation F1	Validation accuracy
Random Forest	0.9735	0.9110	89.56%
Histogram Gradient Boosting	0.9717	0.9049	88.91%
Logistic Regression	0.9583	0.8729	85.33%

Random Forest has the highest validation average precision and is selected before calibration. These values describe uncalibrated candidates on the validation partition. They are not the held-out performance of the deployed calibrated model.

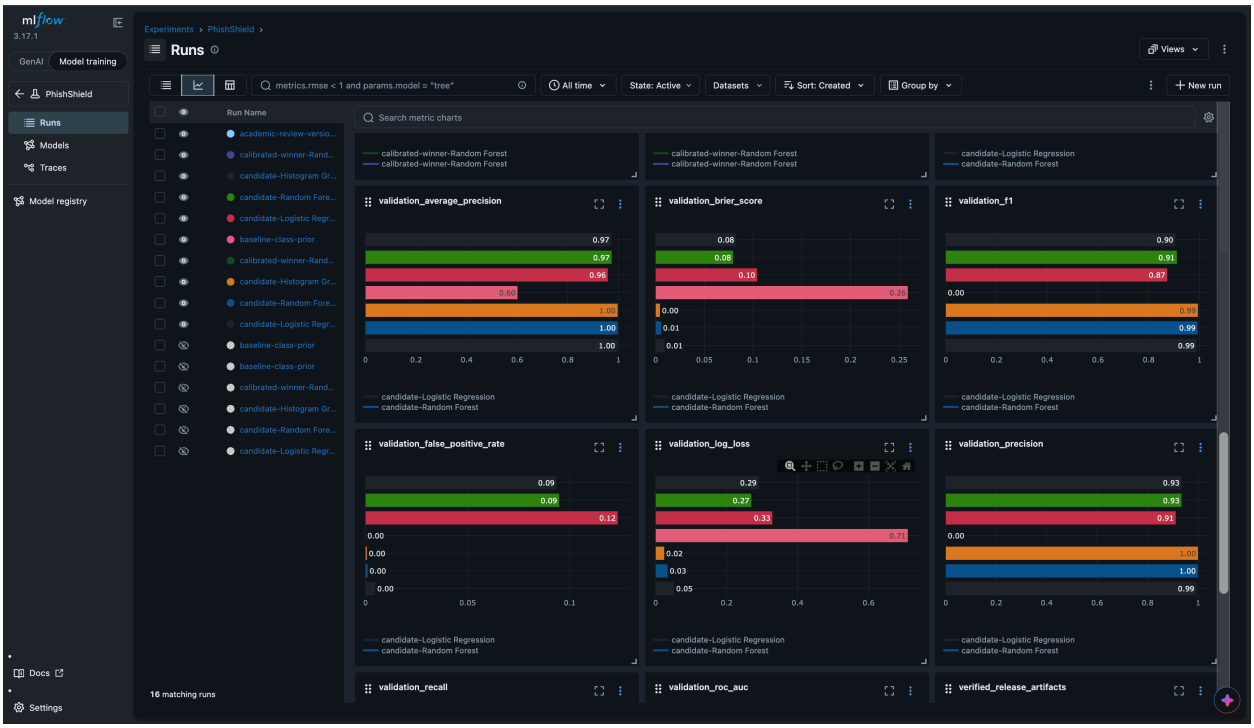


Figure 8.1 MLflow candidate and historical run comparisons

The MLflow screenshot contains runs from more than one experiment cycle. Higher historical values are visible alongside the current corpus results. The table above uses the current deployment training cycle only; comparing runs from different datasets as if they were the same benchmark would be misleading.

Table 8.2 Final calibrated model on 1,944 held-out URLs

Held-out metric	Value
Accuracy	82.00%
Phishing precision	86.49%
Phishing recall	75.80%
Phishing F1 score	80.79%
Average precision	0.9078
ROC AUC	0.9066
False positive rate	11.82%
Brier score	0.1281
Log loss	0.4070

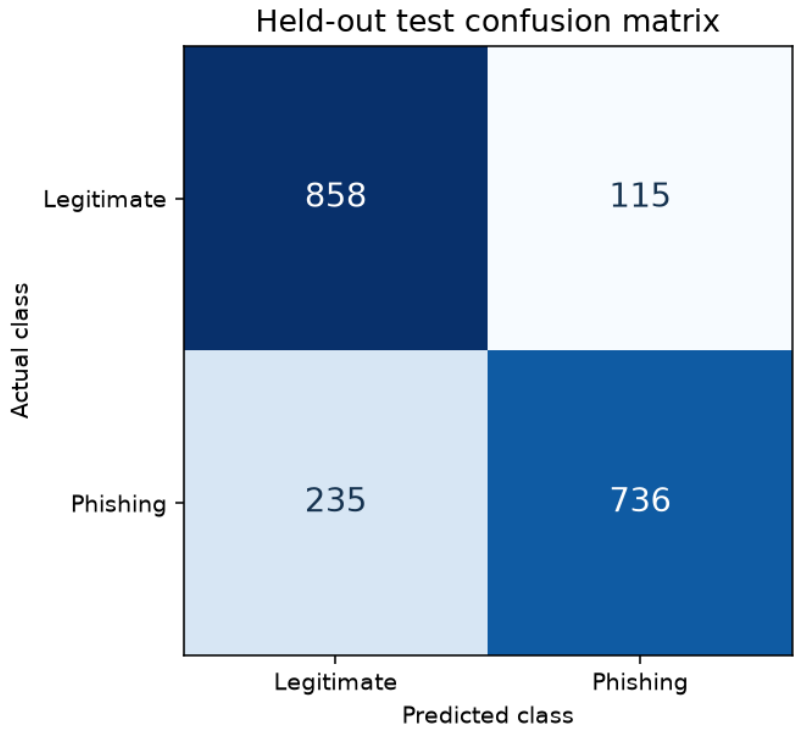


Figure 8.2 Held-out confusion matrix for the deployed release

The model correctly classifies 858 legitimate and 736 phishing URLs. It flags 115 legitimate URLs and misses 235 phishing URLs at the 0.5 threshold. Recall of 75.80% therefore leaves a meaningful false-negative limitation. Domain-warning rules are excluded from these classifier metrics.

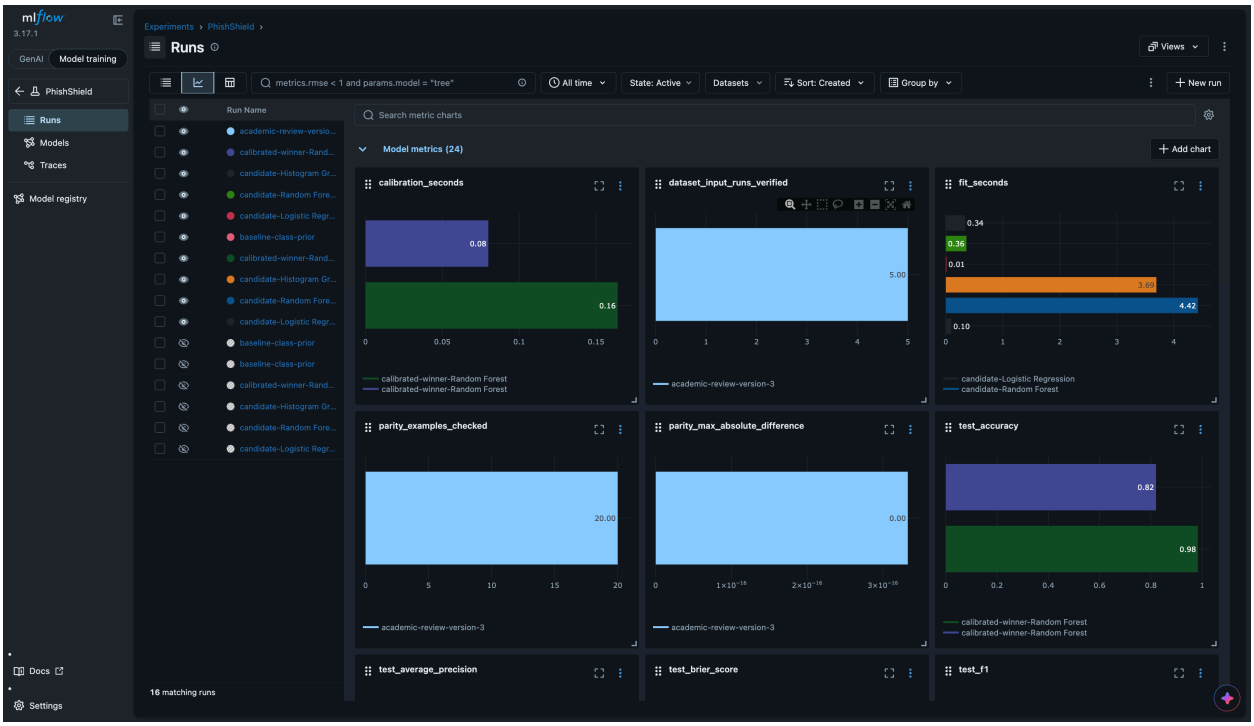


Figure 8.3 MLflow verification metrics and historical accuracy runs

The academic review screenshot records five verified dataset-input runs and a twenty-example parity check. These concern the local registry/export review. They are separate from the later 11,980-case native-to-browser test, whose maximum probability difference is 3.33×10^{-16} and whose maximum feature difference is zero.

The screenshot also shows approximately 0.98 accuracy for an older run and 0.82 for the current corpus. The source project records that the older UCI experiment had a restrictive legitimate-URL format, masking false positives on ordinary paths. The current release uses the more varied Hannousse–Yahiouche corpus. These are different cohorts, so the two scores do not measure a like-for-like increase or decrease in quality.

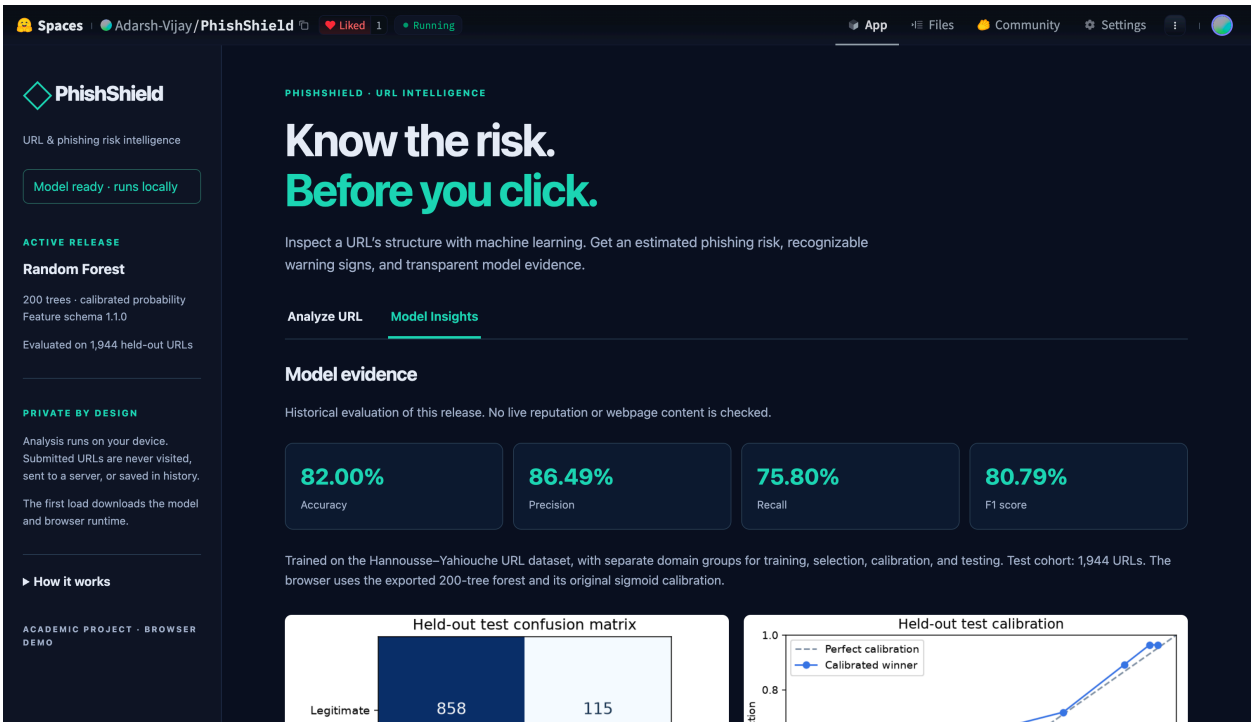


Figure 8.4 Model Insights on the running Hugging Face Space

The deployed Model Insights view displays the same rounded held-out metrics as Table 8.2 and identifies the 1,944-URL test cohort. This connects the public interface to documented evaluation evidence. The page also states that no live reputation or webpage content is checked.

Probability calibration supports interpretable estimates on the historical evaluation distribution. It does not guarantee calibration on current traffic. Data drift, adversarial changes and different class prevalence can change performance. The parity test shows that the deployment preserves the reference behavior on its fixtures; it does not prove that every possible URL will be classified correctly.

CHAPTER 9

CHALLENGES FACED

9.1 Dataset Bias and Generalization

The earlier dataset made legitimate examples unusually easy to distinguish by URL format. Reviewing realistic benign paths exposed false positives that its headline accuracy concealed. The current release uses a more varied corpus, separates domain groups across partitions and reports the lower measured result. Fresh data and time-based evaluation are still needed to assess current attacks.

9.2 Deployment Constraints

The initial Docker hosting route was unavailable for the account during deployment. The application was therefore converted to a Static Space. This required exporting the forest and calibrator, preserving parsing behavior and packaging the required suffix resources. Running the original Python logic through Pyodide reduced the risk of inconsistent URL handling.

9.3 Permissions and Integration

The connected Hugging Face repository access was read-only, so publishing used the authorized browser session. The deployed application serves its model files directly and does not require access tokens in browser code. Local Jenkins configuration also required a workspace-contained controller and plugin cache; this keeps its files separate from the machine’s default application state.

9.4 Verification and Interpretation

A release can load successfully and still compute different outputs. The export was therefore checked against complete feature vectors, model outcomes and error messages. Another challenge was explaining why a lookalike domain can deserve high caution even when the classifier score is low. The interface preserves the original probability and explains the independent warning rather than presenting a modified score as a model result.

CHAPTER 10

CONCLUSION

PhishShield delivers a working URL risk analyzer with reproducible model development and a public browser deployment. The system combines shared feature extraction, comparison of three classifiers, separate calibration, domain-disjoint testing and release verification. The selected Random Forest achieves 82.00% accuracy and an 80.79% phishing F1 score on 1,944 held-out URLs.

The deployed Static Space preserves the tested Python behavior. All 11,980 parity cases pass, including invalid inputs, with only floating-point-scale differences in probability. The browser performs inference locally and does not contact submitted websites. The local Streamlit application remains available for model review and demonstration.

The project also demonstrates the engineering workflow around the model. Jira records assigned tasks and review state, GitHub maintains source code, MLflow records model evidence, and Jenkins executes tests, checks release integrity, verifies the export and packages the result. A controlled failure demonstrates that the pipeline blocks downstream stages when a check fails.

The principal limitations are the historical dataset, missed phishing examples, the limited domain-warning catalogue and the absence of content or live reputation evidence. These limitations define the system’s role as an academic decision aid. The report’s measured results support the working implementation while leaving current-web effectiveness and production-scale operation for future evaluation.

CHAPTER 11

FUTURE ENHANCEMENTS

11.1 Evaluation on Newer Data

Collect a licensed and independently labeled contemporary URL corpus, retain collection dates and evaluate a later time window separately. Report precision and recall by URL format, domain type and attack family. Threshold selection should reflect the intended false-positive and false-negative costs and use validation data only.

11.2 Release Automation and Quality

Connect Jenkins to reviewed repository changes, preserve immutable build inputs and add an authorized deployment stage with restricted credentials. Keep model training, approval and publication as distinct decisions. Static analysis, dependency scanning and a deliberate rollback procedure would strengthen release governance. Container builds should be tested before a Docker deployment is claimed.

11.3 Runtime and Monitoring

Benchmark browser cold-start time, memory and per-URL latency on several devices. Reduce initial asset size where prediction parity can be maintained. Monitor public asset availability and release identity without collecting submitted URLs. Kubernetes or autoscaling would be relevant to a future server-backed design and are not required for the present client-side inference path.

11.4 Detection and Usability

Expand and evaluate the domain-warning catalogue with labeled examples, including internationalized names. Optional reputation or content evidence could add coverage, but would require a new privacy and execution design. Accessibility testing and clearer uncertainty explanations could improve usability. Each extension should preserve a distinction between model output, supplementary evidence and verified ground truth.

REFERENCES

[1] PhishShield project team. PhishShield source repository. GitHub.
https://github.com/Adarshvijaycx/PhishShield

[2] A. Hannousse and S. Yahiouche. Web page phishing detection. Mendeley Data, version 3, 2021. DOI 10.17632/c2gw7fy2j4.3. CC BY 4.0.
https://data.mendeley.com/datasets/c2gw7fy2j4/3

[3] scikit-learn developers. Probability calibration. scikit-learn documentation.
https://scikit-learn.org/stable/modules/calibration.html

[4] MLflow contributors. MLflow Tracking. Official documentation.
https://mlflow.org/docs/latest/ml/tracking/

[5] Hugging Face. Static HTML Spaces. Hugging Face Hub documentation.
https://huggingface.co/docs/hub/spaces-sdks-static

[6] GitHub. Building and testing Python. GitHub Actions documentation.
https://docs.github.com/en/actions/tutorials/build-and-test-code/python

[7] Jenkins contributors. Pipeline. Jenkins User Handbook.
https://www.jenkins.io/doc/book/pipeline/

[8] Pyodide contributors. Using Pyodide in a web worker. Official documentation.
https://pyodide.org/en/stable/usage/webworker.html

[9] PhishShield project team. Public application and deployment assets. Hugging Face Spaces.
https://huggingface.co/spaces/Adarsh-Vijay/PhishShield

Online references accessed 15 October 2026. The dataset mirror revision and release identifiers are recorded in Appendix F. Screenshots in this report were captured on 15 October 2026; the Jenkins view records the pipeline configured for this report.

APPENDICES

Appendix A Jira Workflow and Team

The observed workflow is To Do → In Progress → In Review → Done. Work items include repository verification (SCRUM-6), pipeline review (SCRUM-7) and prediction/error testing (SCRUM-8). Evidence of completion should be attached when each item meets its acceptance criteria; the screenshots preserve the work state at capture time.

Table A.1 Project team

Team member	Registration number
Vaishnavi	RA2311056030134
Manisha Kumari	RA2311056030187
Adarsh Vijay	RA2311056030190
Prince Panwar	RA2311056030142

Appendix B Git and Reproduction Commands

Run the commands from a trusted local checkout. Training creates local model and MLflow artifacts; it is separate from the already verified public deployment.

```
git clone https://github.com/Adarshvijaycx/PhishShield.git
cd PhishShield
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.lock
.venv/bin/python -m pip install --no-deps -e .
.venv/bin/phishshield download
.venv/bin/phishshield train
.venv/bin/phishshield verify
.venv/bin/phishshield serve
```

Start the local experiment viewer separately with .venv/bin/phishshield mlflow. On Windows, use the environment's Scripts directory in place of bin. Use the current checked-out Jenkinsfile and CI helpers when reproducing the local pipeline.

Appendix C Jenkinsfile and Pipeline

The Declarative Jenkinsfile defines six stages and a post-build block. It uses an existing locked Python environment, copies the supplied source and validates the static model without retraining it. Publishing to Hugging Face is not part of this job.

Table C.1 Jenkins pipeline stages

Stage	Implemented action
Prepare source	Copy source to the Jenkins workspace, excluding local caches, tracking state and raw data.
Environment and syntax	Run Python version check, pip check and compileall; apply the optional negative control.
Unit and integration tests	Run pytest and create reports/pytest.xml.
Verify trained release	Run phishshield.cli verify against artifacts/release.
Static prediction parity	Run ci/verify_static.py against the static model and reference fixtures.
Package verified release	Create the release ZIP and a per-file SHA-256 inventory.
Post build	Publish JUnit output and archive the generated reports and package.

Key commands executed within the copied project:

```
"$PYTHON_BIN" -m pip check
"$PYTHON_BIN" -m compileall -q src app.py
"$PYTHON_BIN" -m pytest -q --junitxml=reports/pytest.xml
"$PYTHON_BIN" -m phishshield.cli verify \
  --release artifacts/release
"$PYTHON_BIN" ci/verify_static.py \
  "$STATIC_DIR" reports/static-parity.json
"$PYTHON_BIN" ci/package_static.py \
  "$STATIC_DIR" reports/PhishShield-static-verified.zip
```

Appendix D Verification Records

Table D.1 Verification evidence

Check	Recorded outcome
Local Python suite	217 passed; 38 warnings; 51.43 seconds.
Local release verification	Model checksum and 20 examples passed.
Live browser comparison	11,980 passed; 0 failed; feature error 0; maximum probability error ≈ 3.33×10^{-16}.
Hosted assets	Six core files matched local SHA-256 values.
Jenkins negative control	Build #1: intentional FAILURE after environment checks; later stages skipped; 3.517 s.
Jenkins normal verification	Build #2: SUCCESS; 217 tests passed (18.76 s), 20 release examples and 11,980 parity cases verified; total 30.137 s.

A warning count is not a test failure. The browser comparison includes both expected acceptance and expected rejection. The sample establishes agreement for the tested inputs at a tolerance of 10^{-12}; it is not an exhaustive proof over all possible URLs.

Appendix E Screenshot Index

Figures 5.1 and 5.2 show Jira work tracking. Figure 5.3 shows the Jenkins pipeline. Figures 6.1 and 6.2 show the public input interface and local Streamlit result. Figures 8.1 and 8.3 show MLflow comparison and verification views. Figure 8.4 shows the deployed model evidence. The screenshots are contextual evidence; the numeric result tables use the saved release artifacts.

The original screenshots contain multiple historical runs and different registry identities. Their captions explain these distinctions. Figures 4.1 and 4.2 are architectural diagrams of the implementation, and Figure 8.2 is the trained release's confusion matrix.

Appendix F Code and Release Identity

The browser evaluator traverses each exported tree using the reference feature order, averages the phishing leaf probabilities and applies the original sigmoid calibration. The following pseudocode summarizes the model calculation; validation and warning handling remain separate.

```
features = extract_features(parse_url(input_url))
x = convert_each_feature_to_float32(features)
raw = mean(tree_leaf_probability(tree, x)
           for tree in forest)
probability = 1 / (1 + exp(A * raw + B))
model_class = "phishing" if probability >= 0.5 else "legitimate"
score = 100 * probability
warnings = independent_domain_checks(input_url)
```

For the deployed export, A = −6.391012050590151 and B = 3.2573404896784126. The model contains 200 trees and uses feature schema 1.1.0. The browser uses Python 3.13.2 through Pyodide 0.29.3.

Table F.1 Reproducibility identifiers

Identifier	Value
Source revision	7662eb555af538a40a7aa0202f689f0f1752699b
Space revision	f61a924031b3b236051a554a94e57cb4e7270ab5
Deployment training run	a79c5a87dfdf4d73b144c6df61cb9190
Dataset mirror revision	1375d450d4540205ac11781b807249c9509739cd
Model SHA-256	8c1407b76ab0f0ee61b64b2f76d0ccce44e9341ccaac0de3fdc9019945786c5c

Code is MIT licensed. The Hannousse–Yahiouche dataset retains CC BY 4.0 attribution. The local source project's registry version 3 and the deployment training database's version 1 are different registry identities; the identifiers above describe the deployed export.